

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Activity / Exp No.: Project-Based Learning (PBL) Part 1

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Application-Level Custom Logging Middleware and Route Dispatching in Express.js

2. AIM

To design, implement, and verify an Express.js HTTP web application incorporating custom application-level middleware to intercept, timestamp, and log incoming client requests (HTTP method and target URL) before delegating execution to designated route endpoints ('/', '/about', '/students').

3. OBJECTIVE

- To understand the architectural role of Middleware functions in Express.js as interceptors in the HTTP request-response cycle.
- To implement a custom logging middleware function receiving (req, res, next) arguments.
- To capture real-time execution timestamps using `new Date().toLocaleString()`.
- To inspect request metadata including HTTP method (`req.method`) and pathname (`req.url`).
- To understand and utilize the `next()` callback function to pass execution control to subsequent middleware or matching route handlers.
- To mount application-level middleware globally across all routes using `app.use(logger)`.
- To configure multiple HTTP GET route endpoints ('/', '/about', '/students') and verify end-to-end request processing in local browser environment.

4. THEORY

Express.js is fundamentally a routing and middleware web framework whose core operation is structured as a pipeline of middleware functions. Middleware functions are functions that have access to the request object (`req`), the response object (`res`), and the `next` function in the application's request-response cycle. These functions can execute arbitrary code, make modifications to request and response objects, end the request-response cycle, or pass control to the next middleware in the execution stack.

A standard middleware signature comprises three parameters: `function(req, res, next)`. The '`req`' object encapsulates client request data (headers, query parameters, URL path, HTTP method), while '`res`' represents the HTTP response stream. The third argument, '`next`', is a function provided by Express that, when invoked, tells the routing engine that current middleware processing is complete and execution should proceed to the next matching middleware or route handler. If `next()` is not called, the request hangs indefinitely.

4. THEORY (Continued)

Application-level middleware is bound directly to an instance of the app object using `app.use()` or `app.METHOD()`. By invoking `app.use(logger)` without specifying a mount path, the middleware becomes a global interceptor executed for every single incoming HTTP request regardless of the HTTP method or URL path. This is the foundation for cross-cutting concerns such as request logging, authentication/authorization checks, body parsing, CORS headers, and security hardening.

In production web architectures, request logging provides observability, audit trails, and performance monitoring. By recording timestamps, HTTP methods (GET, POST), and request URLs, developers can track usage patterns, diagnose routing failures, and trace API client interactions. The separation of concerns between middleware (logging) and route handlers (business logic) promotes clean, modular, and maintainable software architecture.

5. ALGORITHM / PROCEDURE

1. Open a terminal and navigate to the project directory (PBL/Middleware).
2. Initialize a Node.js project manifest using `npm init -y`.
3. Install Express.js framework using `npm install express`.
4. Create the server entrypoint file named `server.js`.
5. Import Express using `const express = require('express')`.
6. Instantiate the Express application using `const app = express()`.
7. Define custom middleware function `logger(req, res, next)` with 3 parameters.
8. Obtain the current formatted timestamp using `new Date().toLocaleString()`.
9. Log timestamp, `req.method`, and `req.url` to standard output using `console.log()`.
10. Invoke `next()` to pass execution control to subsequent route handlers.
11. Mount the logger middleware globally using `app.use(logger)`.
12. Configure GET `'/'` route returning 'Welcome to the Express.js application!'.
13. Configure GET `'/about'` route returning 'This is the About page.'.
14. Configure GET `'/students'` route returning 'Students page'.
15. Bind server to port 3000 using `app.listen(3000, callback)` and log startup URL.
16. Execute `node server.js` in terminal to start the HTTP listener.
17. Navigate in a browser to `http://localhost:3000/`, `/about`, and `/students`.
18. Verify web browser responses and terminal logging output with timestamps.

6. PROGRAM

server.js

```
const express = require("express");

const app = express();

// Middleware for logging incoming requests
function logger(req, res, next) {
  const time = new Date().toLocaleString();

  console.log(
    `${time} - ${req.method} ${req.url}`
  );

  // Move to the next middleware or route
  next();
}

// Use the logging middleware for all requests
app.use(logger);

// Home route
app.get("/", (req, res) => {
  res.send("Welcome to the Express.js application!");
});

// About route
app.get("/about", (req, res) => {
  res.send("This is the About page.");
});

// Students route
app.get("/students", (req, res) => {
  res.send("Students page");
});

// Start the server
app.listen(3000, () => {
  console.log("Server is running on http://localhost:3000");
});
```

package.json

```
{
  "name": "middleware",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

7. CODE EXPLANATION

Code Snippet	Detailed Technical Explanation
<code>const express = require("express");</code>	Imports Express web framework module providing HTTP server abstraction and middleware routing.
<code>const app = express();</code>	Instantiates Express application object to manage middleware registration and route definitions.
<code>function logger(req, res, next) { ... }</code>	Declares custom middleware accepting request (req), response (res), and next handler pointer (next).
<code>const time = new Date().toLocaleString();</code>	Generates a localized timestamp string representing the exact time request entered the server.
<code>console.log(`\${time} - \${req.method}...`);</code>	Formats and outputs request auditing data (timestamp, HTTP verb, and target pathname) to terminal.
<code>next();</code>	Passes control to the next middleware function or matching route handler in execution pipeline.
<code>app.use(logger);</code>	Registers logger as application-level middleware executed globally for all incoming client requests.
<code>app.get("/", (req, res) => { ... });</code>	Configures GET route handler for root endpoint ("/") responding with a plain-text welcome message.
<code>app.get("/about", ...); app.get("/students")</code>	Defines secondary GET routes serving specific page content for informational endpoints.
<code>app.listen(3000, callback)</code>	Binds Express server to port 3000 on localhost and begins listening for client socket connections.

8. TEST CASES AND EXECUTION DATA

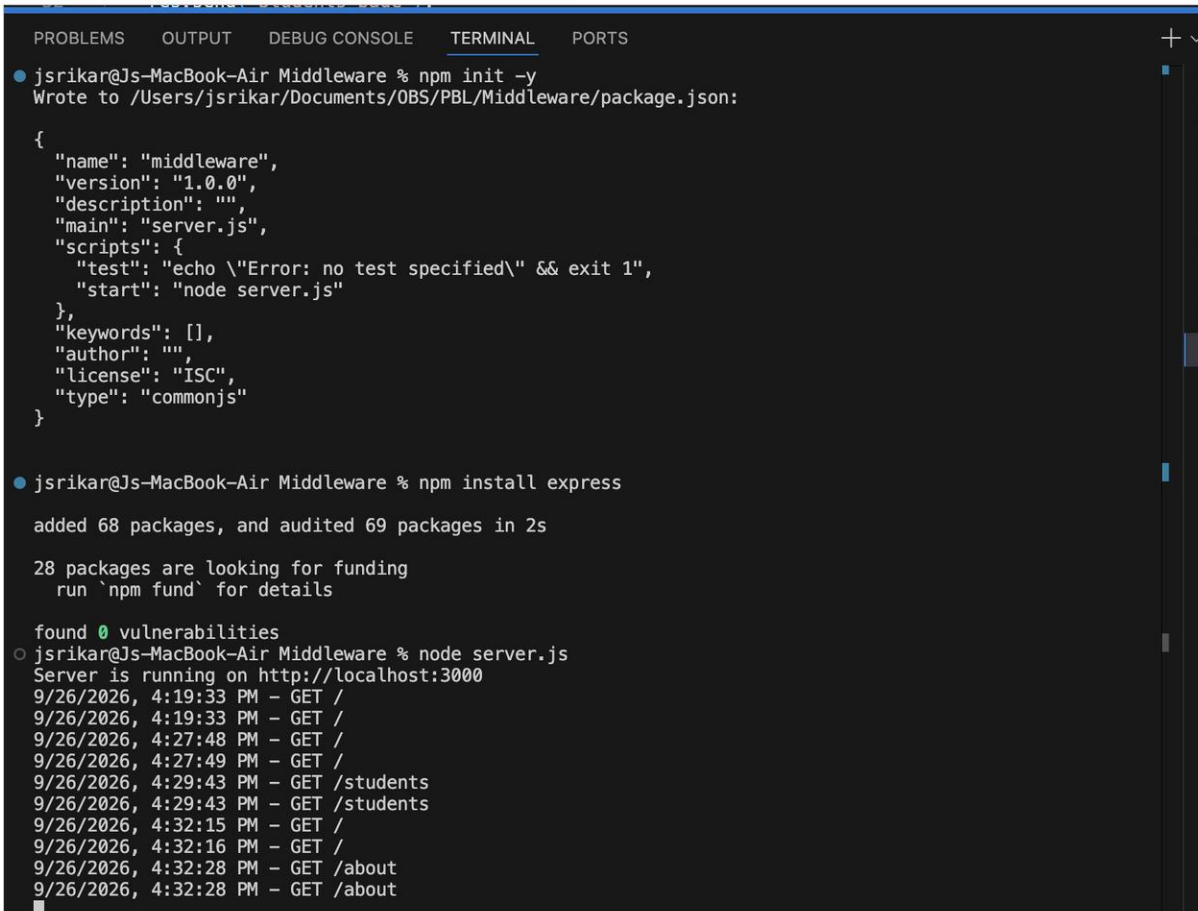
Test Case	Target	Input / Action	Description	Expected Result	Actual Result	Status
TC-01	Root Route	GET /	Request root home page via browser	Welcome message rendered; GET / logged	Welcome text rendered; GET / logged	Pass
TC-02	About Route	GET /about	Request about page via browser	About text rendered; GET /about logged	About text rendered; GET /about logged	Pass
TC-03	Students Route	GET /students	Request students route via browser	Students page rendered; /students logged	Students page rendered; /students logged	Pass
TC-04	Middleware Chain	next() invocation	Verify pipeline execution continuity	Control passes to route without hang	All routes responded; zero timeouts	Pass
TC-05	Server Binding	Port 3000	Start server via node server.js	Server listens on port 3000	Logged: Server is running on port 3000	Pass

9. OUTPUT

Output 1: Execution Event & Console Trace

```
[Event 1] Launched application: 'node server.js' on port 3000.
[Event 2] Initialized logger middleware: function(req, res, next) mounted via app.use().
[Event 3] Registered routes: GET '/', GET '/about', GET '/students'.
[Event 4] Client GET '/': logger intercepted -> Logged: '9/26/2026, 4:19:33 PM - GET /'.
[Event 5] Route handler executed: 200 OK -> Text: 'Welcome to the Express.js application!'.
[Event 6] Client GET '/about': logger intercepted -> Logged: '9/26/2026, 4:32:28 PM - GET /about'.
[Event 7] Route handler executed: 200 OK -> Text: 'This is the About page.'.
[Event 8] Client GET '/students': logger intercepted -> Logged: '9/26/2026, 4:29:43 PM - GET /students'.
[Event 9] Route handler executed: 200 OK -> Text: 'Students page'.
[Event 10] Server maintains active listener loop on port 3000.
```

Output 2: Interactive Terminal Workflow & Request Logging



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● jsrikar@Js-MacBook-Air Middleware % npm init -y
Wrote to /Users/jsrikar/Documents/OBS/PBL/Middleware/package.json:

{
  "name": "middleware",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

● jsrikar@Js-MacBook-Air Middleware % npm install express

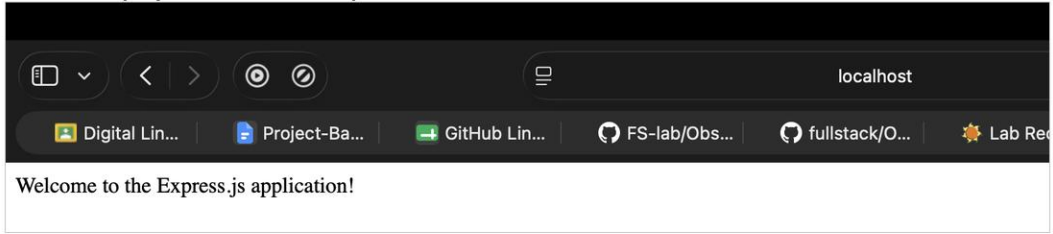
added 68 packages, and audited 69 packages in 2s

28 packages are looking for funding
  run `npm fund` for details

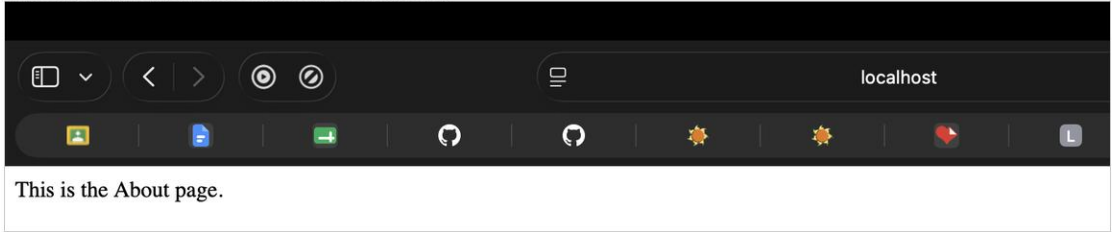
found 0 vulnerabilities
○ jsrikar@Js-MacBook-Air Middleware % node server.js
Server is running on http://localhost:3000
9/26/2026, 4:19:33 PM - GET /
9/26/2026, 4:19:33 PM - GET /
9/26/2026, 4:27:48 PM - GET /
9/26/2026, 4:27:49 PM - GET /
9/26/2026, 4:29:43 PM - GET /students
9/26/2026, 4:29:43 PM - GET /students
9/26/2026, 4:32:15 PM - GET /
9/26/2026, 4:32:16 PM - GET /
9/26/2026, 4:32:28 PM - GET /about
9/26/2026, 4:32:28 PM - GET /about
```

9. OUTPUT (Continued)

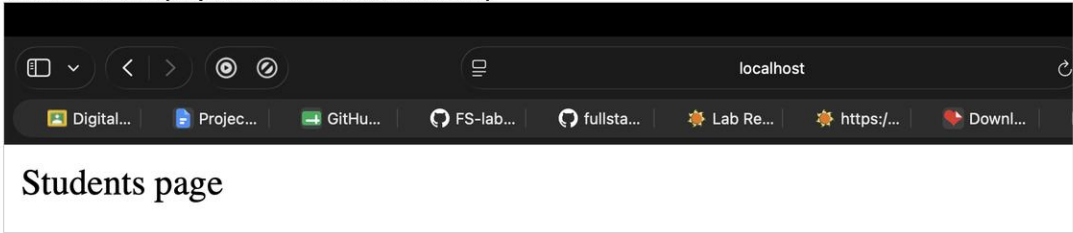
Output 3a: Home Route (http://localhost:3000/)



Output 3b: About Route (http://localhost:3000/about)



Output 3c: Students Route (http://localhost:3000/students)



10. RESULT

Thus, an Express.js web server featuring custom application-level logging middleware was successfully designed, implemented, and verified across all route endpoints ('/', '/about', '/students'). Every incoming HTTP request was properly intercepted, timestamped, and logged to stdout before delivering expected HTTP responses.

Verification Status: Completed & Tested Successfully

Execution Engine: Node.js (V8) | Framework: Express.js (Middleware)

Faculty Signature: _____